

ROYAUME DU MAROC

ÉCOLE NATIONALE SUPÉRIEURE DES MINES DE RABAT

SERVICE DES STAGES

Dossier technique

Technologies, services, variables d'environnement, certificats et configurations particulières

Sommaire

Objet du document	3
1. Technologies utilisées	3
1.1 Vue d'ensemble	3
1.2 Développement	4
1.3 Intégration et déploiement	5
1.4 Organisation du code	5
2. Services indispensables au fonctionnement	7
2.1 Services automatiques	7
2.2 Services externes	7
2.3 Opérations manuelles	8
2.4 Vérifier que tout tourne	9
3. Variables d'environnement	10
3.1 Emplacement et principe	10
3.2 Plateforme des conventions de stage	10
3.3 Plateforme des visites d'entreprise	11
3.4 Appliquer une modification	11
3.5 Fichier .env complet	12
4. Certificats SSL	13
4.1 Emplacement des fichiers	13
4.2 Installation et renouvellement	14
4.3 Configuration nginx	15
5. Configurations particulières	17
5.1 Le schéma ne s'exécute qu'une fois	17
5.2 up -d et non restart	17
5.3 Reconstruction sans cache	17
5.4 Redémarrer nginx après reconstruction	18
5.5 Taille des envois sur deux niveaux	18
5.6 Synchronisation de l'heure	18
5.7 Messagerie sur le port 587	19
Récapitulatif des commandes	20

Objet du document

Ce dossier réunit les informations nécessaires à l'exploitation et à la reprise des deux plateformes du Service des Stages. Il s'adresse à un informaticien n'ayant pas participé au développement.

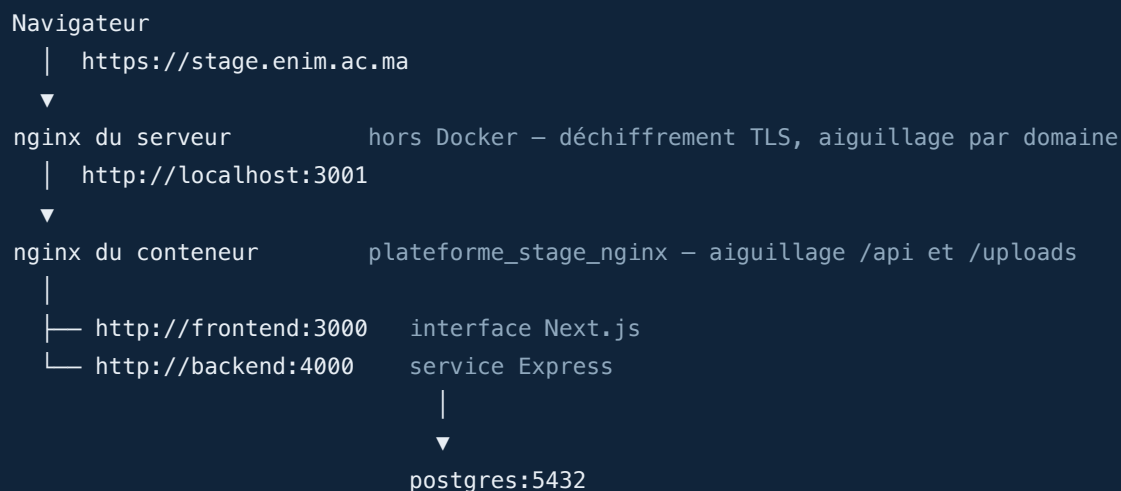
Les guides de l'étudiant et de l'administrateur, distincts, traitent de l'usage courant des applications. Le présent document ne s'intéresse qu'à ce qui les fait fonctionner.

1. Technologies utilisées

1.1 Vue d'ensemble

Les deux plateformes suivent une architecture identique et restent totalement indépendantes : bases de données distinctes, comptes distincts, cycles de mise à jour distincts. Elles cohabitent sur la même machine sans partager aucune ressource applicative.

Chaque plateforme repose sur quatre conteneurs, orchestrés par Docker Compose. En amont, un serveur nginx installé directement sur la machine assure le chiffrement et oriente les requêtes vers la bonne plateforme selon le nom de domaine demandé.



Rôle	Conventions de stage	Visites d'entreprise	Port
Entrée	plateforme_stage_nginx	visite_ensmr_nginx	80
Interface	plateforme_stage_frontend	visite_ensmr_frontend	3000
Traitement	plateforme_stage_backend	visite_ensmr_backend	4000
Base	plateforme_stage_db	visite-ensmr-postgres-1	5432

Sur la machine hôte, la plateforme des conventions est publiée sur le port `3001` et celle des visites sur le port `3002`. Ces ports ne sont utilisés que par le nginx du serveur ; ils restent joignables directement à des fins de diagnostic, mais contournent alors le chiffrement et la limite de taille des envois.

1.2 Développement

Couche	Technologie	Remarques
Interface	Next.js 14, React 18	Routage par répertoire <code>pages</code> . Compilation à la construction de l'image
Mise en forme	Bootstrap 5	Complété par des styles en ligne et, pour la barre de navigation, des styles à portée limitée
Traitement	Express 5 sur Node 20	Image de base <code>node:20-alpine</code>
Base de données	PostgreSQL 15	Accès par la bibliothèque <code>pg</code> , requêtes paramétrées
Authentification	jsonwebtoken, bcrypt	Jeton de 8 heures. Mots de passe chiffrés, jamais stockés en clair
Connexion Google	google-auth-library	Plateforme des conventions uniquement
Courriers	nodemailer	SMTP <code>smtp.gmail.com</code> , port 587, STARTTLS
Documents PDF	Puppeteer, PDFKit	Chromium installé dans l'image du service de traitement
Fichiers déposés	multer	10 Mo par fichier, PDF uniquement
Tableurs	xlsx	Import des inscrits et des filières, export des demandes
Archives	archiver	Téléchargement groupé des documents d'une année
Protections	helmet, express-rate-limit, cors	En-têtes de sécurité, 10 tentatives d'authentification par quart d'heure

POIDS DES IMAGES

L'image du service de traitement embarque Chromium, nécessaire à la production des documents PDF. Elle pèse plusieurs centaines de mégaoctets et sa reconstruction complète prend plusieurs minutes. C'est normal, et il ne faut pas l'interrompre.

1.3 Intégration et déploiement

Le code est versionné dans deux dépôts Git distincts. Le déploiement consiste à récupérer la dernière version sur le serveur, reconstruire les images concernées, puis recréer les conteneurs.

```
cd ~/plateforme-stage-ensmr
git pull
sudo docker compose build --no-cache backend frontend
sudo docker compose up -d
sudo docker restart plateforme_stage_nginx
```

Ne reconstruire que ce qui a changé : `backend` pour une modification du traitement, `frontend` pour l'interface. L'interruption de service est de l'ordre de la minute et les données ne sont pas affectées.

Les trois particularités de cette séquence — l'option `--no-cache`, l'emploi de `up -d` plutôt que `restart`, et le redémarrage du nginx interne — sont expliquées au chapitre 5. Elles ne sont pas facultatives.

Persistance des données

Les données survivent à toute reconstruction, y compris à une suppression complète des conteneurs. Elles résident dans des volumes Docker indépendants.

Volume	Contenu
<code>postgres_data</code>	Intégralité de la base de données
<code>uploads_data</code>	Pièces déposées et conventions produites
<code>assets_data</code>	Ressources de mise en page des conventions

LA COMMANDE À NE JAMAIS LANCER

`docker compose down -v` supprime les volumes, donc la base de données et tous les documents déposés. L'opération est irréversible. La forme `docker compose down`, sans l'option finale, est en revanche sans danger.

1.4 Organisation du code

Les deux dépôts suivent la même organisation : l'interface occupe la racine, le service de traitement le sous-répertoire `backend`.

```

plateforme-stage-ensmr/
├─ pages/                interface : une page par fichier
│  └─ etudiant/          espace étudiant
│     └─ admins/         espace administrateur
├─ components/           éléments réutilisables
├─ nginx/nginx.conf      configuration du serveur d'entrée
├─ next.config.js        redirections /api et /uploads
├─ Dockerfile            image de l'interface
├─ docker-compose.yml    description des quatre conteneurs
├─ .env                  configuration (non versionné)
└─ backend/
   └─ index.js            authentification, demandes, imports Excel
   └─ schema.sql          création des tables
   └─ migrations/        évolutions du modèle de données
   └─ middleware/auth.js  vérification du jeton et des rôles
   └─ routes/             points d'accès par domaine
   └─ utils/              envoi de courriers, production des PDF
   └─ data/               fichiers de référence au format CSV

```

Fichier	Domaine couvert
index.js	Connexion, inscription, mot de passe oublié, dépôt d'une demande, imports Excel
routes/admin-demande.js	Instruction des demandes, validation, refus, avis pédagogique, convention, exports
routes/admin-annulation.js	Instruction des demandes d'annulation
routes/etudiant-annulation.js	Dépôt d'une demande d'annulation
routes/etudiant-statut.js	Suivi des dossiers et téléchargement de la convention
routes/documents.js	Dépôt et contrôle des pièces de fin de stage
routes/archives.js	Consultation et suppression par année
routes/eligibilite.js	Vérification de l'éligibilité d'une adresse

La plateforme des visites, plus restreinte, regroupe ses points d'accès dans `backend/index.js` et `backend/routes/visites.js`. Ses utilitaires `generateNoticePdf.js` et `generateEmargementPdf.js` produisent respectivement la note de visite et la feuille d'émargement.

2. Services indispensables au fonctionnement

2.1 Services automatiques

Ces services démarrent avec la machine et se relancent seuls. Aucune intervention n'est requise après un redémarrage du serveur.

Service	Rôle	Vérification
docker	Exécute les huit conteneurs des deux plateformes	<code>systemctl is-enabled docker</code>
nginx	Chiffrement TLS et aiguillage par nom de domaine	<code>systemctl status nginx</code>
chrony	Synchronisation de l'horloge, dont dépend la connexion Google	<code>chronyc tracking</code>
Conteneurs	Interfaces, traitements et bases de données	<code>docker ps -a</code>
cron	Déclenche la sauvegarde quotidienne à deux heures	<code>sudo crontab -l</code>

Les conteneurs portent la directive `restart: unless-stopped` dans les deux fichiers `docker-compose.yml`. C'est elle qui garantit leur redémarrage automatique après une coupure de courant ou un redémarrage du serveur. Elle ne doit pas être retirée.

ORDRE DE DÉMARRAGE

La base de données porte un contrôle de santé, et le service de traitement attend qu'elle réponde avant de démarrer. Après un redémarrage du serveur, compter une à deux minutes avant que l'ensemble soit disponible.

2.2 Services externes

Les plateformes dépendent de trois services extérieurs à la machine. La fermeture de l'un d'eux dégrade le fonctionnement sans nécessairement provoquer d'erreur visible.

Service	Flux requis	Conséquence d'une interruption
Messagerie Gmail	587/TCP sortant	Plus aucun courrier n'est envoyé. Les opérations aboutissent malgré tout : l'envoi est indépendant du traitement
Serveurs de temps	123/UDP sortant	L'horloge dérive. Au-delà de quelques minutes d'écart, la connexion Google cesse de fonctionner
Identité Google	443/TCP sortant	La connexion par compte Google échoue. La connexion par mot de passe reste disponible

Le service de messagerie exige en outre un **mot de passe d'application** valide, distinct du mot de passe du compte Google. Il est révoqué dès que le mot de passe principal change.

2.3 Opérations manuelles

Ces opérations n'ont aucun automatisme et relèvent d'une décision humaine. Les trois premières incombent au service des stages, les deux dernières à l'administrateur.

Opération	Fréquence	Conséquence d'un oubli
Import de la liste des inscrits	Chaque rentrée	Aucun étudiant ne peut créer de compte ni se connecter
Import des filières	À chaque évolution	Filières obsolètes proposées, ou adresses de diffusion périmées
Archivage annuel	Fin d'année	Saturation progressive de l'espace disque
Renouvellement du certificat	Avant le 6 février 2027	Les navigateurs refusent l'accès aux deux plateformes
Copie externe des sauvegardes	Périodique	Une défaillance du disque emporterait les données et leurs sauvegardes

SAUVEGARDE AUTOMATIQUE

Une sauvegarde quotidienne est en place. Le script `/home/stage/sauvegarde.sh` s'exécute chaque nuit à deux heures, via la table des tâches programmées de l'administrateur. Il produit un export de chacune des deux bases et une archive des documents déposés, dans `/home/stage/sauvegards/`, et supprime automatiquement ce qui dépasse trente jours.

```
sudo crontab -l | grep sauvegarde
ls -lh ~/sauvegards/
```

Une exécution manuelle reste possible à tout moment par `sudo /home/stage/sauvegarde.sh`. Le journal se consulte dans `/home/stage/sauvegarde.log`.

CES SAUVEGARDES RESTENT SUR LA MÊME MACHINE

Elles protègent d'une fausse manœuvre ou d'une suppression accidentelle, mais non d'une défaillance matérielle : le disque qui porterait la panne emporterait aussi les sauvegardes. Une copie vers un support distinct reste à organiser avec le service informatique.

Par ailleurs, une restauration doit être testée au moins une fois. Une sauvegarde dont on n'a jamais vérifié la restauration ne garantit rien.

```
cat ~/sauvegards/plateforme-stage-ensmr_<date>.sql \
| sudo docker exec -i plateforme_stage_db psql -U <utilisateur> -d <base>
```

2.4 Vérifier que tout tourne

```
sudo docker ps -a
```

Les huit conteneurs doivent porter la mention `Up`. La mention `Exited` signale un arrêt, et le nombre entre parenthèses en donne la raison : `0` pour un arrêt volontaire, toute autre valeur pour une interruption anormale.

Un neuvième élément, extérieur à Docker, conditionne l'accès : le nginx du serveur. S'il est arrêté, aucune des deux adresses ne répond alors même que tous les conteneurs fonctionnent. Le cas se reconnaît à ce que les plateformes restent joignables par `IP:3001` et `IP:3002`.

```
sudo systemctl status nginx --no-pager
timedatectl
sudo docker exec plateforme_stage_backend nc -zv smtp.gmail.com 587
```

3. Variables d'environnement

3.1 Emplacement et principe

Chaque plateforme lit un fichier `.env` placé à la racine de son répertoire, aux côtés du `docker-compose.yml`. Ce fichier n'est pas versionné : il contient les mots de passe et doit être créé à l'installation à partir du modèle `.env.example` fourni.

```
~/plateforme-stage-ensmr/.env
~/visite-ensmr/.env
```

Une variable n'atteint le conteneur que si elle est **déclarée dans le `docker-compose.yml`**, dans la section `environment` du service concerné. Une ligne ajoutée au seul `.env` reste sans effet.

3.2 Plateforme des conventions de stage

Variable	Rôle
POSTGRES_USER	Utilisateur de la base de données
POSTGRES_PASSWORD	Mot de passe de la base. Fixé à la création du volume, sa modification ultérieure n'a pas d'effet rétroactif
POSTGRES_DB	Nom de la base de données
JWT_SECRET	Clé de signature des jetons d'authentification. Sa modification déconnecte tous les utilisateurs. Doit être longue et aléatoire, et différente de celle de l'autre plateforme
FRONTEND_URL	Adresse publique, en <code>https://</code> . Sert au contrôle des origines autorisées et à la construction des liens contenus dans les courriers. Laisser en <code>http://</code> , elle fait rejeter toutes les requêtes de l'interface
EMAIL_USER	Adresse d'expédition des courriers
EMAIL_PASSWORD	Mot de passe d'application Google, seize caractères sans espaces. Ce n'est pas le mot de passe du compte
SERVICE_STAGES_EMAIL	Destinataire des notifications de nouvelle demande de convention
GOOGLE_CLIENT_ID	Identifiant client Google. Vide, la connexion par compte Google est désactivée sans autre conséquence
EMAIL_OVERRIDE	Redirige la totalité des courriers vers une adresse unique. Réservé aux tests

EMAIL_OVERRIDE DOIT RESTER VIDE EN EXPLOITATION

Renseignée, cette variable détourne tous les envois vers l'adresse indiquée : ni les étudiants, ni les listes de diffusion, ni la direction ne reçoivent quoi que ce soit. Le sujet des messages détournés porte le préfixe [TEST → destinataire] , et le service de traitement l'inscrit dans ses journaux à chaque envoi.

3.3 Plateforme des visites d'entreprise

Variable	Rôle
POSTGRES_USER	Utilisateur de la base de données
POSTGRES_PASSWORD	Mot de passe de la base
POSTGRES_DB	Nom de la base de données
JWT_SECRET	Clé de signature des jetons. Distincte de celle de l'autre plateforme
FRONTEND_URL	Adresse publique, en <code>https://</code>
EMAIL_USER	Adresse d'expédition des courriers
EMAIL_PASSWORD	Mot de passe d'application Google, propre à cette plateforme
SERVICE_STAGES_EMAIL	Destinataire des notifications de nouvelle visite
CC_VISITE	Adresses de la direction mises en copie des notes de visite, séparées par des virgules
EMAIL_OVERRIDE	Redirige la totalité des courriers vers une adresse unique. Réservé aux tests

`CC_VISITE` et `SERVICE_STAGES_EMAIL` possèdent une valeur de repli inscrite dans le code. Laisseres vides, elles n'empêchent pas le fonctionnement. Pour connaître les adresses réellement utilisées :

```
sudo docker exec visite_ensmr_backend sh -c 'echo "CC_VISITE : [${CC_VISITE}]"; grep -A1 "CC_VISITE |" /app/routes/visites.js'
```

3.4 Appliquer une modification

```
cd ~/plateforme-stage-ensmr
nano .env
sudo docker compose up -d
```

UP -D, ET NON RESTART

Les variables sont fixées à la création du conteneur. `docker compose restart` relance le conteneur existant avec ses anciennes valeurs et ne relit pas le fichier `.env` : la modification paraît alors sans effet, ce qui conduit à chercher la cause ailleurs. Seul `docker compose up -d` recrée le conteneur avec la nouvelle configuration.

Pour vérifier qu'une valeur est bien parvenue au conteneur, sans afficher un secret :

```
sudo docker exec plateforme_stage_backend printenv FRONTEND_URL
sudo docker exec plateforme_stage_backend sh -c 'echo "${#EMAIL_PASSWORD} caractères"'
```

3.5 Fichier `.env` complet

Modèle commenté pour la plateforme des conventions de stage :

```
# Base de données
POSTGRES_USER=plateforme
POSTGRES_PASSWORD=<mot de passe long et aléatoire>
POSTGRES_DB=plateforme_stage

# Authentification – openssl rand -base64 48
JWT_SECRET=<chaîne aléatoire, différente de celle des visites>

# Adresse publique
FRONTEND_URL=https://stage.enim.ac.ma

# Messagerie – mot de passe d'application, pas celui du compte
EMAIL_USER=stage@anim.ac.ma
EMAIL_PASSWORD=<seize caractères, sans espaces>
SERVICE_STAGES_EMAIL=stage@anim.ac.ma

# Connexion Google
GOOGLE_CLIENT_ID=<identifiant client OAuth 2.0>

# Tests uniquement – doit rester absent en exploitation
# EMAIL_OVERRIDE=test-stage@anim.ac.ma
```

Et pour la plateforme des visites d'entreprise :

```
# Base de données
POSTGRES_USER=visite
POSTGRES_PASSWORD=<mot de passe long et aléatoire>
POSTGRES_DB=visite_ensmr

# Authentification
JWT_SECRET=<chaîne aléatoire, différente de celle des stages>

# Adresse publique
FRONTEND_URL=https://visite.enim.ac.ma

# Messagerie
EMAIL_USER=stage@enim.ac.ma
EMAIL_PASSWORD=<seize caractères, propre à cette plateforme>
SERVICE_STAGES_EMAIL=stage@enim.ac.ma
CC_VISITE=directeur@enim.ac.ma,direction.pedagogique@enim.ac.ma,sg@enim.ac.ma,achat@enim.ac.ma,restaurant@enim.ac.ma

# Tests uniquement – doit rester absent en exploitation
# EMAIL_OVERRIDE=test-stage@enim.ac.ma
```

MANIPULATION DES SECRETS

Ces fichiers ne doivent jamais être versionnés ni transmis par messagerie. Les deux plateformes doivent porter des clés `JWT_SECRET` différentes : une clé partagée permettrait d'utiliser un jeton de l'une sur l'autre.

4. Certificats SSL

4.1 Emplacement des fichiers

Le certificat en service est un certificat générique `*.enim.ac.ma` délivré par HARICA, qui couvre les deux plateformes. Il est porté par le nginx du serveur, et non par les conteneurs : les échanges circulent en clair entre ce serveur et les conteneurs, mais uniquement à l'intérieur de la machine.

Fichier	Rôle	Permissions
<code>/etc/ssl/enim/fullchain.pem</code>	Certificat et chaîne de certification	<code>644 root:root</code>
<code>/etc/ssl/enim/privateKey.key</code>	Clé privée. Secret	<code>600 root:root</code>

Ces fichiers sont référencés par les deux configurations de sites, `/etc/nginx/sites-available/stage` et `/etc/nginx/sites-available/visite`, activées par un lien symbolique depuis `/etc/nginx/sites-`

enabled/ .

```
sudo openssl x509 -in /etc/ssl/enim/fullchain.pem -noout -subject -dates
```

Le certificat en service **expire le 6 février 2027**. Passé cette date, les navigateurs afficheront un avertissement de sécurité et refuseront l'accès aux deux plateformes, sans qu'aucune anomalie ne soit visible côté serveur.

4.2 Installation et renouvellement

Le service informatique fournit trois fichiers : le certificat, la chaîne de certification et la clé privée. La chaîne fournie contient parfois des lignes descriptives entre les blocs de certificats ; la commande `awk` ci-dessous ne retient que les blocs utiles.

```
# Sauvegarde de l'existant
sudo cp /etc/ssl/enim/fullchain.pem /etc/ssl/enim/fullchain.pem.bak
sudo cp /etc/ssl/enim/privateKey.key /etc/ssl/enim/privateKey.key.bak

# Installation
sudo mkdir -p /etc/ssl/enim
sudo awk '/BEGIN CERTIFICATE/,/END CERTIFICATE/' Cert_bundle.pem \
| sudo tee /etc/ssl/enim/fullchain.pem > /dev/null
sudo cp privateKey.key /etc/ssl/enim/
sudo chmod 644 /etc/ssl/enim/fullchain.pem
sudo chmod 600 /etc/ssl/enim/privateKey.key
sudo chown root:root /etc/ssl/enim/*

# Vérification puis application
sudo nginx -t && sudo systemctl reload nginx
```

Contrôler que la chaîne comporte bien ses trois certificats, et que la clé correspond au certificat — les deux empreintes doivent être identiques :

```
grep -c "BEGIN CERTIFICATE" /etc/ssl/enim/fullchain.pem

sudo openssl x509 -noout -modulus -in /etc/ssl/enim/fullchain.pem | openssl md5
sudo openssl rsa -noout -modulus -in /etc/ssl/enim/privateKey.key | openssl md5
```

NE JAMAIS RECHARGER SANS AVOIR TESTÉ

Si `nginx -t` signale une erreur, la configuration en service reste active mais un rechargement rendrait les deux plateformes inaccessibles. Les fichiers `.bak` permettent de revenir en arrière.

MÉTHODE RECOMMANDÉE POUR LE RENOUVELLEMENT

Une clé privée ne devrait jamais circuler par messagerie : des copies subsistent dans les boîtes et sur les serveurs de courrier, hors de tout contrôle. La méthode à privilégier consiste à générer la clé sur le serveur et à ne transmettre que la demande de signature, qui n'est pas un secret.

```
sudo openssl req -new -newkey rsa:2048 -nodes \  
-keyout /etc/ssl/enim/privateKey.key \  
-out /etc/ssl/enim/enim.csr \  
-subj "/C=MA/ST=Rabat-Salé-Kénitra/L=Rabat/O=Ecole Nationale Supérieure des Mines  
de Rabat/CN=*.enim.ac.ma"
```

Seul le fichier `.csr` est transmis. Le certificat signé revient en retour, et la clé privée n'a jamais quitté la machine.

4.3 Configuration nginx

Contenu de `/etc/nginx/sites-available/stage` . Celui des visites est identique, au nom de domaine et au port près — `visite.enim.ac.ma` et `localhost:3002` .

```

server {
    listen 80;
    server_name stage.enim.ac.ma;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    http2 on;
    server_name stage.enim.ac.ma;

    ssl_certificate      /etc/ssl/enim/fullchain.pem;
    ssl_certificate_key  /etc/ssl/enim/privateKey.key;
    ssl_protocols        TLSv1.2 TLSv1.3;
    ssl_session_cache    shared:SSL:10m;
    ssl_session_timeout  1d;

    client_max_body_size 50M;

    location / {
        proxy_pass http://localhost:3001;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_cache_bypass $http_upgrade;
    }
}

```

Le premier bloc redirige toute connexion en clair vers la version chiffrée. La directive `client_max_body_size` est indispensable : son absence limiterait les dépôts à un mégaoctet, comme expliqué au point 5.5.

Répercussion sur la configuration des plateformes

Après toute mise en service ou modification du chiffrement, `FRONTEND_URL` doit être en `https://` dans les deux fichiers `.env`, et les origines JavaScript autorisées de l'identifiant client Google mises à jour dans la console. Sans quoi, respectivement, toutes les requêtes de l'interface sont rejetées et la connexion par compte Google cesse de fonctionner.

5. Configurations particulières

Ce chapitre rassemble les comportements qui ne vont pas de soi et qu'il faut connaître avant d'intervenir sur le serveur. Chacun se manifeste par un symptôme qui oriente naturellement vers une fausse piste.

5.1 Le schéma ne s'exécute qu'une fois

Le fichier `backend/schema.sql` est monté dans un répertoire spécial de PostgreSQL, dont le contenu n'est exécuté qu'au tout premier démarrage, sur un volume de données vide. Ensuite, il est ignoré définitivement — heureusement, car il commence par des instructions de suppression de tables.

Toute évolution du modèle de données doit donc être appliquée en deux temps : par une migration sur la base en service, puis reportée dans `schema.sql` pour qu'une réinstallation future reste conforme.

```
sudo docker exec -i plateforme_stage_db psql -U <utilisateur> -d <base> \  
< backend/migrations/001-filieres-reference.sql
```

Les migrations existantes se trouvent dans `backend/migrations/` des deux dépôts. Elles sont écrites pour être rejouables sans risque : une migration déjà appliquée n'a aucun effet la seconde fois. Chaque nouvelle évolution s'ajoute au répertoire sous forme d'un fichier numéroté, et se reporte dans `schema.sql`.

5.2 up -d et non restart

Les variables d'environnement sont fixées à la création du conteneur. `docker compose restart` relance le conteneur existant sans relire le fichier `.env`. Une modification de configuration paraît alors sans effet.

```
sudo docker compose up -d
```

5.3 Reconstruction sans cache

Docker réutilise les couches déjà construites lorsqu'il estime que rien n'a changé. Cette estimation se révèle parfois fausse pour la compilation de l'interface, et la modification semble alors n'avoir aucun effet.

```
sudo docker compose build --no-cache frontend
```

L'option allonge la construction de quelques minutes, mais garantit que le code déployé est bien celui du dépôt.

5.4 Redémarrer nginx après reconstruction

Le nginx de chaque plateforme résout les noms `frontend` et `backend` une seule fois, à son démarrage, et conserve les adresses en mémoire. Reconstruire ces conteneurs leur attribue de nouvelles adresses sur le réseau Docker, et nginx continue d'écrire vers les anciennes.

Le symptôme est une erreur **502 Bad Gateway** alors que tous les conteneurs fonctionnent. Le diagnostic se lit dans `docker ps` : les conteneurs nginx y affichent une ancienneté supérieure aux autres.

```
sudo docker restart plateforme_stage_nginx visite_ensmr_nginx
```

Ajouter cette commande à la fin de chaque séquence de déploiement.

5.5 Taille des envois sur deux niveaux

La directive `client_max_body_size` doit être déclarée **aux deux niveaux** : sur le nginx du serveur et sur celui du conteneur. En son absence, nginx applique une limite d'un mégaoctet et répond `413 Request Entity Too Large`.

Les deux niveaux sont configurés à 50 Mo. Le nginx du serveur étant traversé en premier, c'est sa valeur qui prime : la retirer ferait échouer tout dépôt de plus d'un mégaoctet, quelle que soit la configuration des conteneurs. Un essai mené directement sur `IP:3001` ne révélerait rien, puisqu'il contourne ce niveau.

5.6 Synchronisation de l'heure

La connexion par compte Google repose sur un jeton dont la validité ne tolère que quelques minutes d'écart. Une horloge décalée la fait échouer avec le message `Token used too late`, tout en faussant les dates enregistrées.

```
chronyc sources -v  
chronyc tracking
```

Une source marquée `^*` et des valeurs de `Reach` non nulles attestent du bon fonctionnement.

DEUX RÉGLAGES NÉCESSAIRES SUR CE SERVEUR

Les sources par défaut d'Ubuntu utilisent le protocole NTS, qui négocie sur le port 4460 en TCP — fermé sur le réseau de l'école. Le fichier `/etc/chrony/sources.d/ubuntu-ntp-pools.sources` a donc été basculé sur les sources NTP classiques, qui n'utilisent que le port 123.

Par ailleurs, la directive `makestep 1 3` n'autorise une correction immédiate que lors des trois premières mises à jour après démarrage. Au-delà, un écart important met plusieurs jours à se résorber, pendant lesquels la connexion Google reste inopérante. En cas de décalage constaté :

```
sudo chronyc makestep
```

5.7 Messagerie sur le port 587

Le port 465, initialement retenu, est bloqué sur le réseau de l'école. Les deux plateformes utilisent le port 587 avec STARTTLS, c'est-à-dire `secure: false` dans la configuration de nodemailer — ce qui ne signifie pas une absence de chiffrement, mais un chiffrement négocié après l'établissement de la connexion.

```
sudo docker exec plateforme_stage_backend nc -zv smtp.gmail.com 587
```

La réponse attendue comporte la mention `open` et arrive en moins d'une seconde. Une commande qui reste sans réponse signifie que le port est bloqué.

Si le port répond mais que les envois échouent, l'erreur `535-5.7.8 Username and Password not accepted` dans les journaux désigne un mot de passe d'application invalide — ou une adresse d'expéditeur erronée, Google ne distinguant pas les deux cas.

Récapitulatif des commandes

Objectif	Commande
État des conteneurs	<code>sudo docker ps -a</code>
Démarrer ou recréer une plateforme	<code>sudo docker compose up -d</code>
Arrêter sans perte de données	<code>sudo docker compose down</code>
Reconstruire un composant	<code>sudo docker compose build --no-cache backend</code>
Redémarrer les nginx internes	<code>sudo docker restart plateforme_stage_nginx visite_ensmr_nginx</code>
Journaux	<code>sudo docker logs --tail 50 <nom></code>
Journaux horodatés récents	<code>sudo docker logs -t --since 5m <nom></code>
Accès à la base	<code>sudo docker exec -it <base> psql -U <utilisateur> -d <base></code>
Jouer une migration	<code>sudo docker exec -i <base> psql -U <u> -d <fichier.sql</code>
Sauvegarder la base	<code>sudo docker exec <base> pg_dump -U <u> > sauvegarde.sql</code>
Vérifier une variable	<code>sudo docker exec <nom> printenv <VARIABLE></code>
Tester la messagerie	<code>sudo docker exec <nom> nc -zv smtp.gmail.com 587</code>
Syntaxe nginx du serveur	<code>sudo nginx -t</code>
Appliquer la configuration nginx	<code>sudo systemctl reload nginx</code>
Échéance du certificat	<code>sudo openssl x509 -in /etc/ssl/enim/fullchain.- pem -noout -dates</code>
Sources de temps	<code>chronyc sources -v</code>
Forcer la correction de l'heure	<code>sudo chronyc makestep</code>
Espace disque	<code>df -h</code>
Volumes Docker	<code>sudo docker volume ls</code>

Document établi dans le cadre du stage effectué au Service des Stages de l'École Nationale Supérieure des Mines de Rabat.
Il est complété par le guide de l'étudiant et le guide de l'administrateur.